

Föreläsning 7

Ändrad 15 Oct 17:19

Det finns ytterligare två loopar i C: `for`-loop och `do`-loop.

Egentligen behövs bara `while`-loopen. De andra två looparna finns för att vi ska kunna skriva lite snyggare programtext ibland. Och lite kortare.

do-loopen

Den enda skillnaden mellan `do`-loopen och `while`-loopen är att `do`-loopen testar sitt villkor *efter* att loopkroppen genomlöpts, medan `while`-loopen testar *innan*.

Följande enklast tänkbara exempel demonstrerar skillnaden:

```

1  #include <stdio.h>
2
3  main()
4  {
5      while (1 == 2)
6          printf("while-utskrift\n");
7
8      do {
9          printf("do-utskrift\n");
10     } while (1 == 2);
11 }
```

Här är utskriften:

```

1  do-utskrift
```

Det blir ingen utskrift från `while`-loopen. I bägge looparna är loopvillkoret alltid falskt. Men `do`-loopen hinner inte upptäcka detta förrän han genomlöst kroppen en gång.

Man använder `do`-loopen när man vill att loopkroppen ska genomlöpas *åtminstone* en gång. Några av mina lösningsförslag till [gruppövning 1](#) vore nog snyggare med `do`-loopar än med `while`-loopar.

(Man brukar rama in `do`-loopens kropp med `{}`. Även när kroppen består av en ensam sats.

Då blir det lättare att se att nyckelordet `while` avslutar en `do`-sats. Och inte inleder en `while`-sats.

for-loopen

Följande utskrift har ni sett förut:

```

1  - 1 - 2 - 3 - 4 - 5 - 6 - 7 - 8 - 9 - 10
```

Men den gången skrevs det ut av en `while`-loop:

```

1  #include <stdio.h>
```

```

2
3 void main()
4 {
5     int i;
6
7     i = 1;
8     while (i <= 10) {
9         printf(" - %d", i);
10        i = i + 1;
11    }
12    printf("\n");
13 }

```

Denna gång skrevs det ut av en `for`-loop:

```

1 #include <stdio.h>
2
3 main()
4 {
5     int i;
6
7     for (i = 1; i <= 10; i = i + 1)
8         printf(" - %d", i);
9     printf("\n");
10 }

```

De två programmen är till 100% ekvivalenta, de betyder exakt samma sak.

Men programmet med `for`-loopen ryms på färre rader. Ni ser hur raderna 7 och 10 från `while`-programmet har plockats in i parentesen till `for`.

Detta är också *den enda* nyttan med `for`-loopen.

`for`-loopen tillför alltså ingenting nytt. Den tillhandahåller bara en praktisk möjlighet. Vi paketerar några satser som finns i många loopar på ett enda ställe.

Vi kan rita en mall för vad `for`-loopen betyder:

```

for (FÖRE; TEST; UPPDATERA) {
    KROPP;
}

```

Oavsett vad *FÖRE*, *TEST*, *UPPDATERA* och *KROPP* är för satser, har ovanstående följande betydelse:

```

FÖRE;
while (TEST) {
    KROPP;
    UPPDATERA;
}

```

`for`-loopen är definitivt snyggare än `while`-loopen i exemplet tidigare. Det är precis såna situationer `for`-loopen är gjord för.

Och *UPPDATERA* är mycket ofta just `i = i + 1`.

Andra språk har ofta följande `for`-loop:

```
for i = 1 to 10 do
```

Då kan man inte göra bland annat denna miss

```
for (i = 1; j <= 10; i = i + 1)
```

C-programmeraren måste som alltid vara extra alert...

Ändringsoperatorer

Jo, jag ljög lite nyss. När jag sa "*UPPDATERA* är mycket ofta just `i = i + 1`."

Det är fel. *UPPDATERA* brukar vara `i++`. Det betyder samma sak. Men ingen *äkta* C-programmerare skulle drömma om att skriva `i = i + 1`.

Den som skriver `i = i + 1` har inte fattat någonting om vad C-programmering *är*. C är ett kortfattat språk -- *minimalistiskt*. Det är därför så många älskar C. Här finns inte en bokstav mer än vad som är absolut nödvändigt.

Att då skriva `i = i + 1` när man kan skriva `i++`...

Här är en lista på `x++` och andra praktiska förkortningar i programspråket C. Samtliga används för att ändra värdet på variabeln `x`:

- `x++` betyder `x = x + 1`
- `++x` betyder också `x = x + 1`
- `x--` betyder `x = x - 1`
- `--x` betyder också `x = x - 1`
- `x += y` betyder `x = x + y`
- `x -= y` betyder `x = x - y`
- `x *= y` betyder `x = x * y`
- `x /= y` betyder `x = x / y`
- `x %= y` betyder `x = x % y`

Det finns en skillnad mellan `x++` och `++x`.

Alla satser i C har ett värde. Även satserna i listan ovan. Värdet är *det nya värdet på x*. Med två undantag: `x++` och `x--`.

Bägge har *det gamla* värdet på `x` som värde.

Följande exempel illustrerar skillnaden:

```
i = j = 1;
n = ++i;      /* n = 2 och i = 2 */
m = j++;      /* m = 1 och j = 2 */
```

PS. Satserna `x++`, `x--`, `--x` och `++x` fungerar bara på heltal. DS.

Nästa lektion

